Sebastian Garcia 22291

Hoja de trabajo #2

1. De las RNN a la atención

En un encoder–decoder recurrente clásico, el encoder debe comprimir la secuencia completa en un único vector. Este cuello de botella obliga a conservar en una representación de tamaño fijo información léxica, sintáctica y semántica; al crecer la oración se pierden detalles y el gradiente atraviesa demasiados pasos. Bahdanau et al. [1] sustituyen esa compresión por un contexto diferente en cada paso: se puntúa cada estado del encoder respecto al estado actual del decoder, se normaliza con softmax y se forma una suma ponderada. Así, el decoder recupera directamente las partes pertinentes de la entrada.

La atención aditiva de Bahdanau usa una red pequeña, usualmente $v^T \tanh(W_q q + W_k k)$; relaciona dimensiones diferentes, pero añade proyecciones y una no linealidad. La atención multiplicativa de Luong usa $q^T k$ o $q^T W k$. El producto punto suele ser más eficiente en hardware moderno porque se expresa como multiplicaciones matriciales densas altamente optimizadas [2].

En la analogía de diccionario, la consulta Q expresa lo que se busca, las llaves K describen las entradas y los valores V contienen la información recuperada. La similitud con todas las llaves produce una búsqueda “suave”, es decir, una mezcla ponderada de valores. En self-attention, Q, K y V son proyecciones de la misma secuencia; en cross-attention, las consultas vienen del decoder y las llaves y valores de la salida del encoder.

Al eliminar la recurrencia, el Transformer procesa en paralelo todos los tokens de una capa y reduce a una operación el camino entre posiciones. Esto favorece dependencias largas y evita el orden secuencial obligatorio y el camino recurrente asociado al desvanecimiento del gradiente, aunque no elimina todos los problemas de optimización.

2. Arquitectura Transformer

La atención escalada es $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$. Si las componentes de q y k son independientes, con media 0 y varianza 1, entonces $\text{Var}(q \cdot k) = d_k$. Sin el divisor, los logits crecen, la softmax se satura y sus gradientes disminuyen; dividir por $\sqrt{d_k}$ mantiene una escala aproximadamente constante [3].

En multi-head attention, h cabezas de dimensión dmodel/h aprenden a la vez relaciones locales, sintácticas, posicionales o globales. Con dmodel fijo no cambia sustancialmente el número de parámetros: las proyecciones concatenadas de Q, K, V y salida siguen siendo aproximadamente $4d_{model}^2$; cambia la partición del cálculo.

La atención sin posición no distingue el orden. La codificación sinusoidal suma senos y cosenos de distintas frecuencias, no añade parámetros y puede extrapolar. Los embeddings aprendidos son flexibles, pero se ligan al máximo entrenado. RoPE rota pares de componentes de Q y K según la posición e introduce desplazamientos relativos en el producto punto [4].

Cada bloque combina atención, una red feed-forward por posición, residuales y LayerNorm. Post-LN normaliza después de sumar el residual; Pre-LN lo hace antes de cada subcapa y ofrece una ruta más directa para el gradiente. La atención densa cuesta $O(n^2 d)$ tiempo y $O(n^2)$ memoria. FlashAttention reduce tráfico y memoria sin aproximar, pero conserva cómputo

cuadrático [5]; ventanas cuestan O(nw) sacrificando conexiones globales directas, y atención lineal aproxima o reordena el kernel a cambio de no reproducir exactamente la softmax.

3. Atención en PyTorch y Hugging Face

- F.scaled_dot_product_attention(Q,K,V,attn_mask,dropout_p,is_causal,scale): attn_mask restringe pares, dropout_p regulariza, is_causal aplica la máscara triangular y scale reemplaza $1/\sqrt{d_k}$.
- nn.MultiheadAttention(embed_dim,num_heads,batch_first,kdim,vdim): define la atención multicabeza. En forward, key_padding_mask oculta relleno, attn_mask controla pares, need_weights solicita pesos y average_attn_weights=False conserva cada cabeza.
- nn.TransformerEncoderLayer(d_model,nhead,dim_feedforward,dropout,norm_first): integra atención, MLP, residuales y normalización. norm_first=True selecciona Pre-LN. generate_square_subsequent_mask crea una máscara causal.
- En Hugging Face, output_attentions=True devuelve un tensor por capa de forma batch × cabezas × n × n, después de la softmax [8]. Se utilizó attn_implementation="eager" para materializar las matrices.

4. Metodología y escalamiento

Se fijó la semilla 22291 y se usaron bert-base-uncased y gpt2, ambos con 12 capas y 12 cabezas, en modo evaluación. Se implementó scaled dot-product attention y se comparó con PyTorch: el error máximo fue 0.0. Después se simularon consultas y llaves aleatorias para medir la entropía de softmax y se calculó la memoria de una matriz float32 de 12 cabezas. Para el análisis lingüístico se extrajeron las 144 matrices por modelo, se buscaron patrones por cabeza, se verificó causalidad, se evaluó el contraste pronominal y se calculó entropía normalizada por capa.

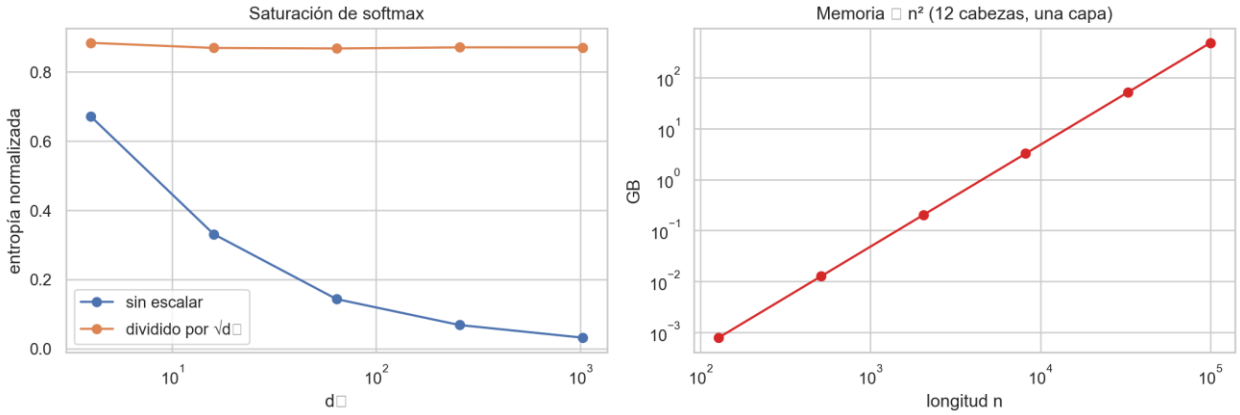


Figura 1. Efecto del escalamiento y crecimiento cuadrático de la memoria.

Sin escalamiento, la entropía normalizada cayó aproximadamente de 0.67 a 0.03 al aumentar d_k de 4 a 1024: la distribución se saturó. Con $1/\sqrt{d_k}$ permaneció cerca de 0.87. La memoria creció cuadráticamente; a 100,000 tokens, únicamente los pesos de una capa y 12 cabezas requieren cerca de 480 GB, sin contar gradientes ni otras activaciones.

Criterios de medición

La especialización se resumió mediante la masa promedio hacia la misma posición, token anterior, token siguiente, puntuación y tokens especiales. Para GPT-2 se comprobó que toda entrada sobre la diagonal fuera cero y se promedió la masa recibida por el primer token. La entropía se normalizó por log n en BERT y por log(i+1) en cada consulta causal de GPT-2, omitiendo la primera posición, cuya entropía es necesariamente cero.

5. Mapas de atención e interpretación

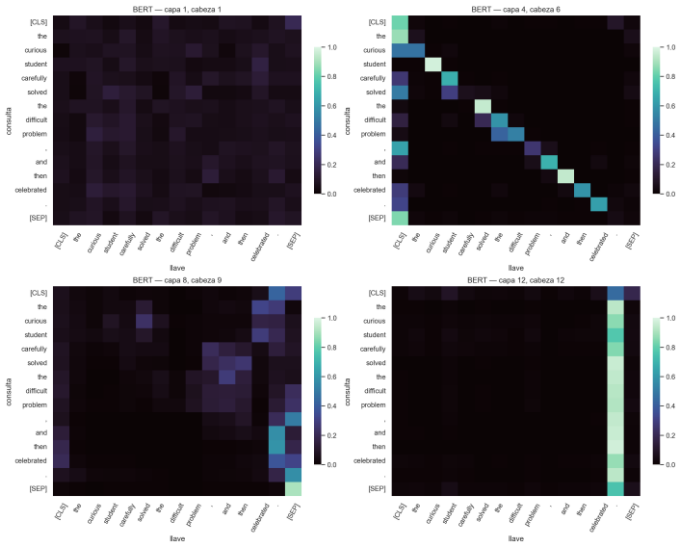


Figura 2. Cuatro cabezas de BERT.

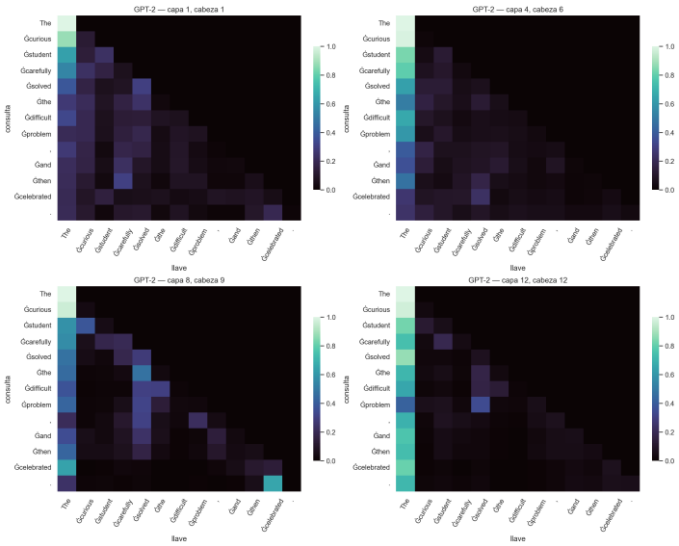


Figura 3. Cuatro cabezas de GPT-2.

BERT

La capa 1/cabeza 1 es relativamente difusa. La capa 4/cabeza 6 muestra una diagonal desplazada y relaciones locales; la capa 8/cabeza 9 concentra masa en puntuación y tokens de cierre; la capa 12/cabeza 12 dirige casi todas las consultas al punto final. El análisis de todas las cabezas confirmó especialización: la mayor masa a la misma posición apareció en capa 12/cabeza 9 (0.429), al token anterior en 7/12 (0.635), al siguiente en 3/10 (0.929), a puntuación en 12/12 (0.870) y a [CLS]/[SEP] en 2/7 (0.917). Los máximos distintos sugieren división funcional, aunque no demuestran que cada cabeza sea indispensable.

GPT-2

Los mapas son estrictamente triangulares inferiores: el mayor peso sobre la diagonal superior fue 0.0. La capa 1/cabeza 1 atiende con fuerza a la posición propia y la capa 4/cabeza 6 al primer token. En capas profundas persiste una columna intensa sobre ese token. Cuantitativamente, la cabeza con mayor masa a la posición propia fue 1/2 (0.968) y la del token anterior 5/12 (0.998); la atención al siguiente token fue 0 por causalidad.

Lectura comparativa

BERT puede usar contexto a ambos lados y por eso aparecen diagonales en ambas direcciones, tokens especiales y puntuación como destinos globales. GPT-2 solo accede al prefijo: la región superior de cada mapa queda vacía y la masa se acumula en posiciones

pasadas. Estas diferencias no son defectos, sino consecuencias de sus objetivos: representación bidireccional en BERT y predicción autorregresiva en GPT-2.

6. Pronombres, attention sinks y entropía

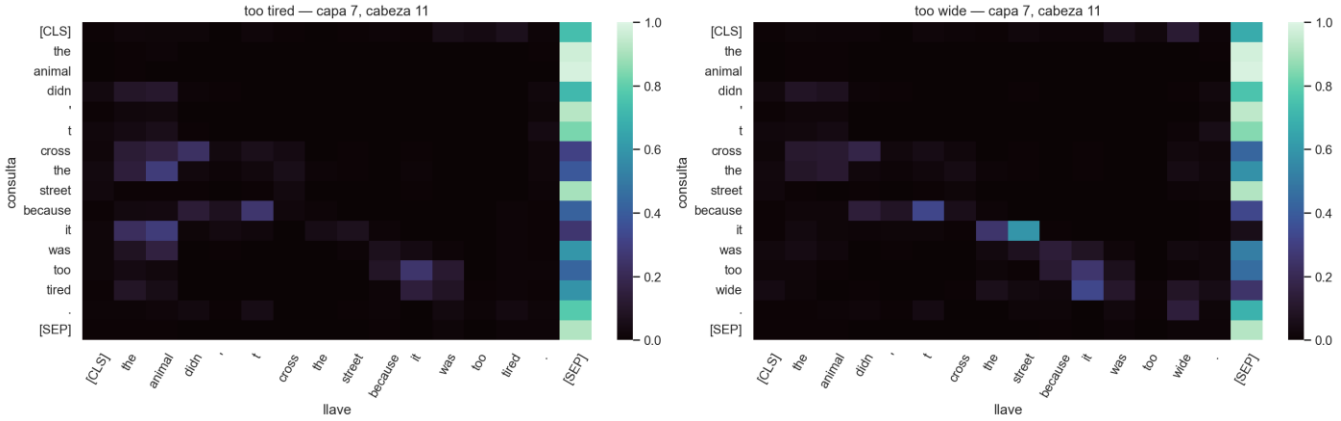


Figura 4. Cabeza seleccionada automáticamente para el contraste de antecedente.

Se evaluaron las 144 cabezas sin selección manual. En la capa 7/cabeza 11, para “too tired”, it asignó 0.2843 a animal y 0.0671 a street; para “too wide”, asignó 0.0119 a animal y 0.5902 a street. Los márgenes fueron 0.2172 y 0.5782. La cabeza cambia de antecedente en la dirección esperada y muestra sensibilidad contextual, aunque un peso aislado no prueba cómo se produjo la decisión final.

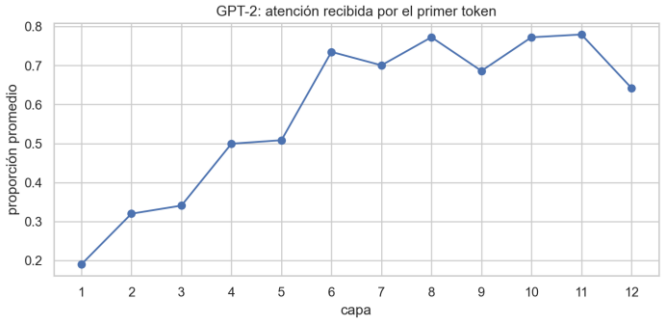


Figura 5. Masa recibida por el primer token.

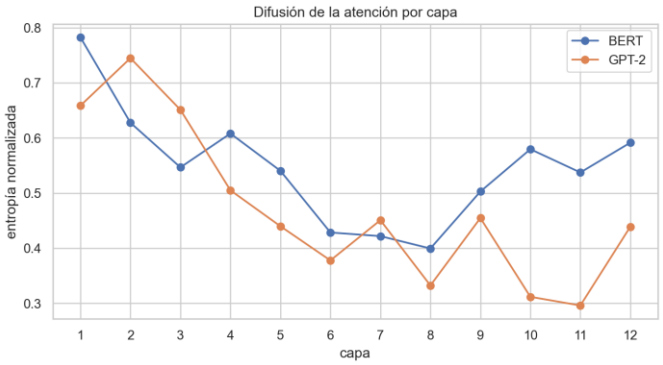


Figura 6. Entropía normalizada por capa.

La atención promedio de GPT-2 al primer token creció de 0.190 en la capa 1 hasta un máximo de 0.780 en la capa 11, evidencia compatible con el attention sink descrito por Xiao et al. [6]: una posición inicial concentra masa aun sin relevancia semántica aparente. La entropía descendió globalmente en ambos modelos. BERT pasó de 0.782 en la primera capa a un mínimo de 0.399 en la 8; GPT-2 alcanzó 0.296 en la 11. Las capas profundas fueron, en conjunto, más concentradas, pero la tendencia no fue monótona y BERT volvió a aumentar al final.

En síntesis, los resultados verifican la implementación numérica, la máscara causal exacta y patrones diferenciados por cabeza. Las visualizaciones permiten formular hipótesis sobre el flujo de información, mientras que los CSV y el resumen JSON conservan las mediciones completas para reproducibilidad.

7. Discusión

La atención permitió entrenar modelos mayores que las RNN porque paraleliza posiciones durante el entrenamiento y acorta a O(1) capas el camino entre tokens, en vez de recorrer O(n) estados. Esto mejora el uso de GPU y el aprendizaje de dependencias largas. Su contrapartida es comparar todos los pares, por lo que tiempo y memoria crecen cuadráticamente.

Los máximos observados para posición propia, vecinos, puntuación y tokens especiales apoyan la especialización, pero no prueban que todas las cabezas sean útiles. Puede existir redundancia; una ablación por cabeza sería necesaria para medir importancia causal. Tampoco los pesos constituyen por sí solos una explicación. Jain y Wallace [7] hallaron baja correlación con importancia por gradiente y distribuciones alternativas con predicciones similares. Wiegreffe y Pinter [9] matizan que la respuesta depende de la definición de explicación y proponen controles uniformes, múltiples semillas y pruebas adversariales. Aquí los mapas son evidencia descriptiva, no atribución causal completa.

Todavía preferiría una RNN para streaming estricto con estado compacto y memoria limitada, o una CNN cuando predominan patrones locales y se busca costo lineal y alto paralelismo. Para documentos de 100,000 tokens no usaría atención densa: consideraría ventanas dispersas con tokens globales, procesamiento jerárquico o atención lineal. FlashAttention mejoraría memoria y velocidad reales, pero no eliminaría por sí sola el número cuadrático de productos.

8. Conclusiones

El escalamiento por $\sqrt{d_k}$ evitó experimentalmente la saturación; BERT exhibió cabezas especializadas y una cabeza sensible al antecedente contextual; GPT-2 cumplió causalidad exacta y mostró un attention sink fuerte. La entropía indicó mayor concentración general en profundidad, con variaciones por arquitectura y capa. La atención mejora el paralelismo y el acceso a dependencias largas, pero introduce un límite cuadrático y sus mapas deben interpretarse como diagnósticos internos, no como explicaciones completas.

Referencias

1. Bahdanau, D., Cho, K. y Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. arXiv:1409.0473.
2. Luong, M.-T., Pham, H. y Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. arXiv:1508.04025.
3. Vaswani, A. et al. (2017). Attention Is All You Need. arXiv:1706.03762.
4. Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.
5. Dao, T. et al. (2022). FlashAttention. arXiv:2205.14135.
6. Xiao, G. et al. (2023). Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453.
7. Jain, S. y Wallace, B. C. (2019). Attention is not Explanation. arXiv:1902.10186.
8. PyTorch, Scaled dot-product attention; Hugging Face, Model outputs. Documentación oficial.
9. Wiegreffe, S. y Pinter, Y. (2019). Attention is not not Explanation. arXiv:1908.04626.

Repositorio: https://github.com/SebastianUVG/HT2_Deep_Learning.git